LaaS Node

Complete Installation & Setup Guide

Bare Metal Ubuntu 22.04 → Fully Validated GPU Session Node

March 2026 | Version 1.0 | Based on POC Runbook v2.0

> **About this Guide:** This is the complete, as-executed installation guide for a LaaS GPU session node. Every command has been run and validated on the POC machine. Commands are presented in exact execution order. All deviations from standard expectations are flagged. This guide takes you from a fresh Ubuntu 22.04 installation to a fully running, multi-session GPU desktop node.

Hardware Reference

Component	POC / Test Node	Production Node (×4)
CPU	AMD Ryzen 9 7950X3D (32T)	AMD Ryzen 9 9950X3D (32T, 3D V-Cache)
GPU	NVIDIA RTX 4090 24GB (Ada, sm_89)	Zotac RTX 5090 32GB (Blackwell, sm_100)
RAM	64GB DDR5	64GB DDR5 G.SKILL 6000MHz
Storage	2TB NVMe (shared with Windows)	Samsung 990 EVO Plus 2TB NVMe (clean)
OS	Ubuntu 22.04 LTS (dual-boot)	Ubuntu 22.04 LTS (clean install)
CUDA arch flag	CUDA_NVRTC_ARCH=89	CUDA_NVRTC_ARCH=100
Total GPU VRAM pool	24GB	32GB

> ⚠**WARNING:** RTX 5090 is Blackwell (sm_100). All validation was on RTX 4090 (Ada, sm_89). Change CUDA_NVRTC_ARCH=89 to CUDA_NVRTC_ARCH=100 in all container

launch commands on production nodes. Re-validate HAMi enforcement and NVENC encoding on first production node before fleet deployment.

STEP 1 BIOS Configuration

Before touching the OS, verify these BIOS settings. On AMD platform (ASUS/MSI/Gigabyte), enter BIOS at POST with DEL or F2.

Setting	Value	Why
AMD-Vi / IOMMU	Enabled	Required for GPU isolation and container GPU passthrough
SVM Mode (AMD-V)	Enabled	Hardware virtualisation — required for KVM/Kata if used later
Resizable BAR / SAM	Enabled	GPU performance — allows CPU full access to GPU memory
XMP / EXPO	Enabled	RAM stability at rated speed (6000MHz)
Above 4G Decoding	Enabled	Required for large GPU BAR mapping
BIOS version	Latest available	Update before OS install — use EZ Flash 3 / M-Flash

ℹ **INFO:** On ASUS boards: AI Overclocking → Enabled for XMP/EXPO. AMD CBS → IOMMU = Enabled. Update BIOS using EZ Flash 3 from USB before starting — do this with Windows still on the machine if dual-booting.

STEP 2 Ubuntu 22.04 Post-Install Baseline

After clean Ubuntu 22.04 installation, run these baseline commands before anything else.

2.1 — System Update

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget git build-essential cmake
sudo apt install -y pkg-config software-properties-common apt-transport-https
sudo apt install -y ca-certificates gnupg lsb-release htop nvtop net-tools
sudo reboot
```

2.2 — Verify Hardware Post-Reboot

```
# Verify CPU
lscpu | grep -E 'Model name|CPU\(s\)|Thread'
```

```
# Verify RAM
free -h

# Verify storage
lsblk
df -h /

# Verify IOMMU is enabled
dmesg | grep -i iommu | head -5
# Expected: AMD-Vi: AMD IOMMUv2 loaded and initialized
```

STEP 3 NVIDIA Driver + CUDA Installation

3.1 — Remove Any Existing NVIDIA Packages

```
sudo apt purge -y nvidia* libnvidia* cuda* 2>/dev/null || true
sudo apt autoremove -y
sudo reboot
```

3.2 — Install NVIDIA Driver 570

```
sudo add-apt-repository ppa:graphics-drivers/ppa -y
sudo apt update
sudo apt install -y nvidia-driver-570 nvidia-utils-570
sudo reboot
```

3.3 — Verify Driver

```
nvidia-smi
# Expected output:
# NVIDIA-SMI 570.x.x   Driver Version: 570.x.x   CUDA Version: 12.8
# GPU 0: NVIDIA GeForce RTX 4090 (or RTX 5090 on production)

# Verify persistence mode
sudo nvidia-smi -pm 1
nvidia-smi | grep Persistence
# Expected: Persistence-M: On
```

3.4 — Install CUDA Toolkit 12.8

```
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-
keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
```

```
sudo apt install -y cuda-toolkit-12-8

# Add CUDA to PATH
echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc
source ~/.bashrc

# Verify
nvcc --version
# Expected: release 12.8
```

> ⚠**WARNING:** nvidia/cuda Docker images (e.g., nvidia/cuda:12.8.0-base) do NOT include nvidia-smi binary. Do not use these images for GPU testing. Use the Selkies image or mount nvidia-smi from host into ubuntu:22.04 for isolated tests.

STEP 4 Docker CE + NVIDIA Container Toolkit

4.1 — Install Docker CE

```
curl -fsSL https://get.docker.com | sudo bash
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker --version
docker run --rm hello-world
```

4.2 — Install NVIDIA Container Toolkit

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
  sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-
toolkit.list | \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-
keyring.gpg] https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# Verify GPU is accessible in a container
docker run --rm --gpus all ubuntu:22.04 bash -c \
  'apt-get install -y pciutils -q 2>/dev/null && lspci | grep -i nvidia'
```

> ✅**VALIDATED:** Container sees GPU confirmed when you see the RTX 4090/5090 in lspci output.

STEP 5 HAMi-Core Build (VRAM + SM Enforcement)

HAMi-core compiles to libvgpu.so — a shared library that intercepts CUDA API calls to enforce per-container VRAM and SM limits. No kernel module, no daemon. Pure LD_PRELOAD interception.

5.1 — Build libvgpu.so

```
cd ~
git clone https://github.com/Project-HAMi/HAMi-core.git
cd HAMi-core
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)

# Verify build
ls -la libvgpu.so
# Expected: ~418784 bytes

# Install to correct path
sudo cp libvgpu.so /usr/lib/libvgpu.so
sudo ldconfig
ls -la /usr/lib/libvgpu.so
```

> **CRITICAL:** Install path MUST be /usr/lib/libvgpu.so — NOT /usr/local/lib/. Wrong path causes silent failures where HAMi loads but LD_PRELOAD is never resolved inside containers.

5.2 — Backup (before any future patching)

```
sudo cp /usr/lib/libvgpu.so /usr/lib/libvgpu.so.backup
ls -la /usr/lib/libvgpu.so.backup
```

> **INFO:** If you later build from the patched fork (loiht2/HAMi-core-fix-memory) for improved nvidia-smi NVML interception, restore from this backup if anything breaks.

STEP 6 CUDA MPS Daemon (Compute Sharing)

MPS (Multi-Process Service) allows multiple containers to share the GPU's SMs concurrently. Required for SM% limits to be meaningful. Must run as a systemd service on the host.

```
sudo tee /etc/systemd/system/cuda-mps.service > /dev/null << 'EOF'
[Unit]
Description=CUDA MPS Control Daemon
After=nvidia-persistenced.service
```

```
[Service]
Type=forking
Environment=CUDA_VISIBLE_DEVICES=0
Environment=CUDA_MPS_PIPE_DIRECTORY=/tmp/nvidia-mps
Environment=CUDA_MPS_LOG_DIRECTORY=/tmp/nvidia-log
ExecStartPre=/usr/bin/nvidia-smi -pm 1
ExecStart=/usr/bin/nvidia-cuda-mps-control -d
ExecStop=/bin/bash -c "echo quit | /usr/bin/nvidia-cuda-mps-control"
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now cuda-mps
sudo systemctl status cuda-mps
# Expected: active (running)
```

> ℹ **INFO:** echo get_server_list | nvidia-cuda-mps-control returning empty is NORMAL. MPS servers spawn on-demand when a CUDA process connects. Not a failure.

STEP 7 ZFS + NFS Persistent User Storage

Each user gets a dedicated ZFS dataset (15GB quota) as their persistent home directory, served via NFS and mounted into containers at /home/ubuntu.

7.1 — Install ZFS and NFS

```
sudo apt install -y zfsutils-linux nfs-kernel-server
```

7.2 — Production: ZFS Pool on Dedicated Partition

```
# On production nodes with dedicated storage partition:
# Replace /dev/nvme0n1p3 with your actual partition
sudo zpool create -f datapool /dev/nvme0n1p3
sudo zfs create datapool/users

# Verify pool
sudo zpool status
sudo zfs list
```

7.3 — POC Only: File-Backed Pool (space constrained)

```
# Only use if no dedicated partition available (POC workaround):
```

```
sudo mkdir -p /vg_containers
sudo truncate -s 20G /vg_containers/nas_pool.img
sudo zpool create -f datapool /vg_containers/nas_pool.img
sudo zfs create datapool/users
```

> ⚠ **WARNING:** File-backed ZFS pool is for POC only. Production must use a dedicated partition for correct performance and reliability.

7.4 — Create User Volumes

```
# Production: 15GB quota per user
# POC: 5GB quota (space constraint)
QUOTA=15G   # change to 5G for POC

# Create test users (replace with real user IDs in production)
for i in 1 2 3 4; do
  sudo zfs create -o quota=$QUOTA datapool/users/testuser$i
done

# Verify quotas
sudo zfs get quota datapool/users/testuser1
```

7.5 — NFS Export and Mount

```
# Configure NFS export
# POC: loopback (127.0.0.1)
# Production: use NAS hostname, not IP
echo '/datapool/users 127.0.0.1(rw,sync,no_subtree_check,no_root_squash)' | \
  sudo tee -a /etc/exports
sudo exportfs -ra
sudo systemctl restart nfs-kernel-server

# Mount NFS
sudo mkdir -p /mnt/nfs/users
sudo mount -t nfs4 127.0.0.1:/datapool/users /mnt/nfs/users

# Make mount persistent
echo '127.0.0.1:/datapool/users /mnt/nfs/users nfs4 defaults 0 0' | \
  sudo tee -a /etc/fstab

# Set ownership (uid 1000 = ubuntu user inside container)
for i in 1 2 3 4; do
  sudo chown -R 1000:1000 /mnt/nfs/users/testuser$i
done

# Verify
ls -la /mnt/nfs/users/
sudo zfs get quota datapool/users/testuser1
```

> ℹ **INFO:** In production: replace 127.0.0.1 with nas.internal hostname. Use abstract user IDs as path components (e.g., /datapool/users/u_8f3a2c) rather than usernames to avoid leaking user identity in df output.

STEP 8 lxcfs (Resource Visibility Spoofing)

lxcfs is a FUSE filesystem that intercepts /proc reads and returns container-cgroup-aware values. Without it, containers see the host's full 64GB RAM and 32-core CPU in free -h, KDE System Settings, and all tools that read /proc.

```
sudo apt install -y lxcfs
sudo systemctl enable --now lxcfs
sudo systemctl status lxcfs
# Expected: active (running)

# Verify all required files are available
ls /var/lib/lxcfs/proc/
# Must see: cpuinfo meminfo stat uptime loadavg diskstats swaps

ls /var/lib/lxcfs/sys/devices/system/cpu/
# Must see: cpu0 cpu1 ... online possible present offline isolated
```

> ℹ **INFO:** lxcfs must be running on the host before launching any containers. Add to all node startup checks. If lxcfs is not running, containers will show host specs to users.

Resource	Without lxcfs	With lxcfs + correct flags	Method
RAM (free -h)	Shows host 62GB	Shows container limit (8GB)	lxcfs /proc/meminfo mount
RAM (KDE System Info)	Shows host 62GB	Shows container limit (8GB)	fake_sysconf.so interceptor (Step 10)
CPU count (nproc)	Shows host 32	Shows container cores (4)	lxcfs /proc/cpuinfo + --cpuset-cpus
CPU (KDE System Info)	Shows host 32 cores	Hidden / correct count	/sys/devices/system/cpu static files (Step 10)
Load average	Shows host load	Shows container load	lxcfs /proc/loadavg mount
Uptime	Shows host uptime	Shows container uptime	lxcfs /proc/uptime mount

STEP 9 Build fake_sysconf.so (RAM Fix for KDE)

KDE reads RAM via sysconf(_SC_PHYS_PAGES) syscall — not /proc/meminfo. lxcfs cannot intercept syscalls, only file reads. This tiny LD_PRELOAD library intercepts the sysconf call and returns the container-correct RAM value.

```
cat > /tmp/fake_sysconf.c << 'EOF'
#define _GNU_SOURCE
#include <unistd.h>
```

```
#include <dlfcn.h>
#include <stdlib.h>
#include <string.h>

// Reads CONTAINER_MEMORY_LIMIT_BYTES env var set at container launch
// Returns correct page count matching the container's RAM limit
long sysconf(int name) {
    static long (*real_sysconf)(int) = NULL;
    if (!real_sysconf)
        real_sysconf = dlsym(RTLD_NEXT, "sysconf");

    if (name == _SC_PHYS_PAGES || name == _SC_AVPHYS_PAGES) {
        const char *limit = getenv("CONTAINER_MEMORY_LIMIT_BYTES");
        if (limit) {
            long page_size = real_sysconf(_SC_PAGESIZE);
            long limit_bytes = atol(limit);
            long pages = limit_bytes / page_size;
            if (name == _SC_AVPHYS_PAGES)
                return (long)(pages * 0.85);
            return pages;
        }
    }
    return real_sysconf(name);
}
EOF

# Compile
gcc -shared -fPIC -o /tmp/fake_sysconf.so /tmp/fake_sysconf.c -ldl

# Install
sudo cp /tmp/fake_sysconf.so /usr/lib/fake_sysconf.so

# Verify
ls -la /usr/lib/fake_sysconf.so
# Expected: ~16K
```

> ℹ **INFO:** CONTAINER_MEMORY_LIMIT_BYTES is passed as an env var at container launch time (e.g., 8589934592 for 8GB). The library reads this at runtime so no recompilation is needed per container.

> □ **CRITICAL:** Do NOT add fake_sysconf.so to selkies-gstreamer's LD_PRELOAD — it causes exit 139 (SIGSEGV). Only load it in the entrypoint (for KDE) and terminal sessions (via bash.bashrc).

STEP 10 Prepare All Host-Side Container Files

These files must exist on the host before launching any container. They are bind-mounted into each container at launch. Run this entire block once per node setup.

10.1 — Pull the Selkies Image

```
docker pull ghcr.io/selkies-project/nvidia-egl-desktop:latest
docker images | grep selkies
# Note: image is Ubuntu 24.04 internally (NOT 22.04)
# Default user inside: ubuntu (uid 1000)
```

10.2 — nvidia-smi Wrapper

```
cat > /tmp/nvidia-smi-wrapper << 'WRAPPER_EOF'
#!/bin/bash
# Reads enforced VRAM limit from CUDA_DEVICE_MEMORY_LIMIT_0 env var
# Replaces total VRAM in nvidia-smi output with container's actual limit

get_vram_limit_mib() {
  if [ -n "$CUDA_DEVICE_MEMORY_LIMIT_0" ]; then
    VAL="${CUDA_DEVICE_MEMORY_LIMIT_0,,}"
    if [[ "$VAL" == *"m" ]]; then
      echo "${VAL//m/}"
    elif [[ "$VAL" == *"g" ]]; then
      echo $(( ${VAL//g/} * 1024 ))
    fi
    return
  fi
  python3 -c "
import struct
with open('/tmp/cudevshr.cache','rb') as f:
    d=f.read(4096)
print(struct.unpack_from('<Q',d,1600)[0]//(1024*1024))
" 2>/dev/null || echo "0"
}

# Pass through non-display commands unchanged
case "$*" in
  *"-pm"*|*"--persistence-mode"*|*"-pl"*|*"--power-limit"*|\
  *"--query-gpu"*|*"--format"*|*"-i "*|*"--id"*)
    exec env -u LD_PRELOAD /usr/bin/nvidia-smi.real "$@"
    ;;
esac

LIMIT_MIB=$(get_vram_limit_mib)
REAL_OUT=$(env -u LD_PRELOAD /usr/bin/nvidia-smi.real "$@" 2>&1)

if [ -n "$LIMIT_MIB" ] && [ "$LIMIT_MIB" -gt "0" ] 2>/dev/null; then
  echo "$REAL_OUT" | sed "s|/ [0-9]*MiB|/ ${LIMIT_MIB}MiB|g"
else
  echo "$REAL_OUT"
fi
WRAPPER_EOF
chmod +x /tmp/nvidia-smi-wrapper
```

10.3 — passwd Wrapper

```
cat > /tmp/passwd-wrapper << 'EOF'
#!/bin/bash
exec env -u LD_PRELOAD /usr/bin/passwd.real "$@"
EOF
chmod +x /tmp/passwd-wrapper
```

10.4 — bash.bashrc with HAMi + sysconf Injection

```
# Get original bashrc from image
docker run --rm --entrypoint cat \
  ghcr.io/selkies-project/nvidia-egl-desktop:latest \
  /etc/bash.bashrc > /tmp/bash.bashrc.orig

# Append both interceptors
cat >> /tmp/bash.bashrc.orig << 'EOF'

# LaaS resource interceptors — applied to all bash terminal sessions
# HAMi: CUDA VRAM/SM limits  |  fake_sysconf: correct RAM in tools
if [ "${HAMI_INJECTED}" != "1" ]; then
  export LD_PRELOAD="/usr/lib/libvgpu.so /usr/lib/fake_sysconf.so"
  export HAMI_INJECTED=1
  export SYSCONF_INJECTED=1
  mkdir -p /tmp/vgpulock 2>/dev/null
fi
EOF
chmod 644 /tmp/bash.bashrc.orig
```

> ⚠ **WARNING:** profile.d scripts are NOT sourced in Konsole — it opens non-login interactive shells. /etc/bash.bashrc is the correct injection point. Do not use profile.d.

10.5 — supervisord Configuration

> ℹ **INFO:** Critical rules: entrypoint gets fake_sysconf.so only (not libvgpu.so — VGL will crash). selkies-gstreamer gets libvgpu.so only (not fake_sysconf.so — segfault exit 139). User terminals get both via bash.bashrc.

```
cat > /tmp/supervisord-hami.conf << 'SUPERVISORD_EOF'
[unix_http_server]
file=/tmp/supervisor.sock
chmod=0700

[supervisord]
logfile=/tmp/supervisord.log
logfile_maxbytes=5MB
logfile_backups=0
loglevel=info
pidfile=/tmp/supervisord.pid
childlogdir=/tmp
nodaemon=true

[rpcinterface:supervisor]
supervisor.rpcinterface_factory = supervisor.rpcinterface:make_main_rpcinterface

[supervisorctl]
serverurl=unix:///tmp/supervisor.sock

[include]
files = /etc/supervisor/conf.d/*.conf

# fake_sysconf.so only — NO libvgpu.so (VGL crash)
[program:entrypoint]
command=bash -c "LD_PRELOAD=/usr/lib/fake_sysconf.so dbus-run-session --
/etc/entrypoint.sh"
```

```
stdout_logfile=/tmp/entrypoint.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=0
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true
priority=1

[program:dbus]
command=bash -c "mkdir -pm700 \"${XDG_RUNTIME_DIR}\"; chown -f \"$(id -nu):$(id -ng)\"
\"${XDG_RUNTIME_DIR}\"; chmod -f 700 \"${XDG_RUNTIME_DIR}\"; dbus-daemon --system --
nofork --nosyslog --nopidfile --address=\"${DBUS_SYSTEM_BUS_ADDRESS}\""
environment=DISPLAY="%(ENV_DISPLAY)s",XDG_RUNTIME_DIR="%(ENV_XDG_RUNTIME_DIR)s",DBUS_SYST
EM_BUS_ADDRESS="%(ENV_DBUS_SYSTEM_BUS_ADDRESS)s"
stdout_logfile=/tmp/dbus.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=0
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true
priority=1

# libvgpu.so only — NO fake_sysconf.so (exit 139 segfault)
[program:selkies-gstreamer]
command=bash -c "if [ \"$(echo ${KASMVNC_ENABLE} | tr '[:upper:]' '[:lower:]')\" !=
\"true\" ]; then LD_PRELOAD=/usr/lib/libvgpu.so /etc/selkies-gstreamer-entrypoint.sh;
else sleep infinity; fi"
stdout_logfile=/tmp/selkies-gstreamer-entrypoint.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=0
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true
priority=20

[program:kasmvnc]
command=bash -c "if [ \"$(echo ${KASMVNC_ENABLE} | tr '[:upper:]' '[:lower:]')\" =
\"true\" ]; then /etc/kasmvnc-entrypoint.sh; else sleep infinity; fi"
stdout_logfile=/tmp/kasmvnc-entrypoint.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=0
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true
priority=20

[program:nginx]
command=bash -c "until nc -z localhost ${SELKIES_PORT:-9081}; do sleep 0.5; done;
/usr/sbin/nginx -g \"daemon off;\""
stdout_logfile=/tmp/nginx.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=0
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
```

```
autorestart=true
priority=30

[group:pipewire-group]
program=pipewire,wireplumber,pipewire-pulse
priority=10

[program:pipewire]
command=bash -c "until [ -S \"/tmp/.X11-unix/X${DISPLAY#*:}\" ]; do sleep 0.5; done;
dbus-run-session -- /usr/bin/pipewire"
environment=PIPEWIRE_LATENCY="128/48000",DISPLAY="%(ENV_DISPLAY)s",DISABLE_RTKIT="y",XDG_
RUNTIME_DIR="%(ENV_XDG_RUNTIME_DIR)s",DBUS_SYSTEM_BUS_ADDRESS="%(ENV_DBUS_SYSTEM_BUS_ADDR
ESS)s",PIPEWIRE_RUNTIME_DIR="%(ENV_XDG_RUNTIME_DIR)s",PULSE_RUNTIME_PATH="%(ENV_XDG_RUNTI
ME_DIR)s/pulse"
stdout_logfile=/tmp/pipewire.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=0
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true

[program:wireplumber]
command=bash -c "until [ \"$(echo ${XDG_RUNTIME_DIR}/pipewire-*.lock)\" !=
\"${XDG_RUNTIME_DIR}/pipewire-*.lock\" ]; do sleep 0.5; done; dbus-run-session --
/usr/bin/wireplumber"
environment=PIPEWIRE_LATENCY="128/48000",DISPLAY="%(ENV_DISPLAY)s",DISABLE_RTKIT="y",XDG_
RUNTIME_DIR="%(ENV_XDG_RUNTIME_DIR)s",DBUS_SYSTEM_BUS_ADDRESS="%(ENV_DBUS_SYSTEM_BUS_ADDR
ESS)s",PIPEWIRE_RUNTIME_DIR="%(ENV_XDG_RUNTIME_DIR)s",PULSE_RUNTIME_PATH="%(ENV_XDG_RUNTI
ME_DIR)s/pulse"
stdout_logfile=/tmp/wireplumber.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=0
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true

[program:pipewire-pulse]
command=bash -c "until [ \"$(echo ${XDG_RUNTIME_DIR}/pipewire-*.lock)\" !=
\"${XDG_RUNTIME_DIR}/pipewire-*.lock\" ]; do sleep 0.5; done; dbus-run-session --
/usr/bin/pipewire-pulse"
environment=PIPEWIRE_LATENCY="128/48000",DISPLAY="%(ENV_DISPLAY)s",DISABLE_RTKIT="y",XDG_
RUNTIME_DIR="%(ENV_XDG_RUNTIME_DIR)s",DBUS_SYSTEM_BUS_ADDRESS="%(ENV_DBUS_SYSTEM_BUS_ADDR
ESS)s",PIPEWIRE_RUNTIME_DIR="%(ENV_XDG_RUNTIME_DIR)s",PULSE_RUNTIME_PATH="%(ENV_XDG_RUNTI
ME_DIR)s/pulse"
stdout_logfile=/tmp/pipewire-pulse.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=0
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true
SUPERVISORD_EOF
```

10.6 — SM% Calculator Script

```
sudo tee /usr/local/bin/calc-sm-percent << 'EOF'
#!/bin/bash
# Usage: calc-sm-percent <container_vram_gb> <total_gpu_vram_gb>
# SM% = VRAM_allocated / total_VRAM * 100 (proportional allocation)
# RTX 4090 POC:  total=24   RTX 5090 Production: total=32
CONTAINER_VRAM=${1:?Usage: calc-sm-percent <container_vram_gb> <total_gpu_vram_gb>}
TOTAL_VRAM=${2:?Usage: calc-sm-percent <container_vram_gb> <total_gpu_vram_gb>}
PCT=$(( (CONTAINER_VRAM * 100) / TOTAL_VRAM ))
[ $PCT -lt 5 ] && PCT=5
echo $PCT
EOF
sudo chmod +x /usr/local/bin/calc-sm-percent

# Verify
echo 'RTX 4090 (24GB pool):'
for v in 4 8 12 16 24; do
  echo "  ${v}GB → $(calc-sm-percent $v 24)% SMs"
done
echo 'RTX 5090 (32GB pool):'
for v in 4 8 16 32; do
  echo "  ${v}GB → $(calc-sm-percent $v 32)% SMs"
done
```

10.7 — Per-Container vgpulock Directories

```
for i in 1 2 3 4; do
  sudo mkdir -p /tmp/vgpulock-$i
  sudo chmod 777 /tmp/vgpulock-$i
done
ls -la /tmp/ | grep vgpulock
```

10.8 — Static CPU Sys Files (per container)

lxcfs does not filter /sys/devices/system/cpu by cpuset. Static files with exact correct cpuset ranges are created and mounted per container. KDE reads these to report CPU count.

```
for i in 1 2 3 4; do
  CPUSET_START=$(( ($i-1)*4 ))
  CPUSET_END=$(( ($i-1)*4+3 ))
  CPUSET="${CPUSET_START}-${CPUSET_END}"
  mkdir -p /tmp/container-$i-cpu
  echo "$CPUSET" > /tmp/container-$i-cpu/online
  echo "$CPUSET" > /tmp/container-$i-cpu/present
  echo "$CPUSET" > /tmp/container-$i-cpu/possible
  echo ""         > /tmp/container-$i-cpu/offline
  echo ""         > /tmp/container-$i-cpu/isolated
  chmod 444 /tmp/container-$i-cpu/*
  echo "Container $i → cpuset: $CPUSET → $(cat /tmp/container-$i-cpu/online)"
done
```

10.9 — Final File Check

```
echo '=== Required host-side files ==='
ls -la /usr/lib/libvgpu.so
ls -la /usr/lib/fake_sysconf.so
ls -la /tmp/nvidia-smi-wrapper
ls -la /tmp/passwd-wrapper
ls -la /tmp/bash.bashrc.orig
ls -la /tmp/supervisord-hami.conf
ls -la /var/lib/lxcfs/proc/meminfo
ls -la /tmp/vgpulock-1 /tmp/vgpulock-2 /tmp/vgpulock-3 /tmp/vgpulock-4
ls -la /tmp/container-1-cpu/online
ls -la /mnt/nfs/users/
echo '=== All good if no errors ==='
```

STEP 11 Launch Containers

11.1 — Port Assignment Scheme

Slot	NGINX_PORT (Web UI)	SELKIES_PORT	METRICS_PORT	DISPLAY	cpuset
1	8081	9081	19081	:20	0-3
2	8082	9082	19082	:21	4-7
3	8083	9083	19083	:22	8-11
4	8084	9084	19084	:23	12-15
N	8080+N	9080+N	19080+N	:19+N	(N-1)*4 to (N-1)*4+3

11.2 — Single Container Launch (Template)

```
# Set variables
i=1
NGINX_P=8081
SELKIES_P=9081
METRICS_P=19081
DISPLAY_N=20
CPUSET=0-3
VRAM_GB=4                              # Adjust per tier
TOTAL_VRAM_GB=24                       # 24 for RTX 4090, 32 for RTX 5090
SM_PCT=$(calc-sm-percent $VRAM_GB $TOTAL_VRAM_GB)
MEM_BYTES=$((8*1024*1024*1024))        # 8GB RAM in bytes
SESSION_HOST="ws-$(openssl rand -hex 4)"
TOKEN=$(openssl rand -hex 16)
echo "URL: http://100.100.66.101:$NGINX_P | token: $TOKEN | host: $SESSION_HOST"

docker run -d \
  --name selkies-$i \
  --hostname $SESSION_HOST \
  --gpus all \
  --cpus=4 \
  --cpuset-cpus=$CPUSET \
  --memory=8g \
```

```
  --ipc=host --network=host --tmpfs /dev/shm:rw \
  -e TZ=UTC \
  -e DISPLAY=:$DISPLAY_N \
  -e DISPLAY_SIZEW=1920 -e DISPLAY_SIZEH=1080 -e DISPLAY_REFRESH=60 \
  -e SELKIES_ENCODER=nvh264enc \
  -e SELKIES_ENABLE_BASIC_AUTH=true \
  -e SELKIES_BASIC_AUTH_PASSWORD=$TOKEN \
  -e NGINX_PORT=$NGINX_P \
  -e SELKIES_PORT=$SELKIES_P \
  -e SELKIES_METRICS_HTTP_PORT=$METRICS_P \
  -e PASSWD=mypasswd \
  -e CUDA_VISIBLE_DEVICES=0 \
  -e CUDA_NVRTC_ARCH=89 \
  -e __NV_PRIME_RENDER_OFFLOAD=1 \
  -e __GLX_VENDOR_LIBRARY_NAME=nvidia \
  -e CUDA_DEVICE_MEMORY_LIMIT_0=${VRAM_GB}192m \
  -e CUDA_DEVICE_SM_LIMIT=$SM_PCT \
  -e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT="0=${VRAM_GB}G" \
  -e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=$SM_PCT \
  -e CUDA_MPS_ENABLE_PER_CTX_DEVICE_MULTIPROCESSOR_PARTITIONING=1 \
  -e CONTAINER_MEMORY_LIMIT_BYTES=$MEM_BYTES \
  -e SELKIES_TURN_HOST=<YOUR_TURN_SERVER_IP> \
  -e SELKIES_TURN_PORT=3478 \
  -e SELKIES_TURN_USERNAME=<TURN_USER> \
  -e SELKIES_TURN_PASSWORD=<TURN_PASS> \
  -e SELKIES_TURN_PROTOCOL=tcp \
  -v /mnt/nfs/users/testuser$i:/home/ubuntu \
  -v /usr/lib/libvgpu.so:/usr/lib/libvgpu.so \
  -v /usr/lib/fake_sysconf.so:/usr/lib/fake_sysconf.so \
  -v /tmp/vgpulock-$i:/tmp/vgpulock \
  -v /usr/bin/nvidia-smi:/usr/bin/nvidia-smi.real \
  -v /tmp/nvidia-smi-wrapper:/usr/bin/nvidia-smi \
  -v /usr/bin/passwd:/usr/bin/passwd.real \
  -v /tmp/passwd-wrapper:/usr/bin/passwd \
  -v /tmp/supervisord-hami.conf:/etc/supervisord.conf \
  -v /tmp/bash.bashrc.orig:/etc/bash.bashrc \
  -v /var/lib/lxcfs/proc/cpuinfo:/proc/cpuinfo:ro \
  -v /var/lib/lxcfs/proc/meminfo:/proc/meminfo:ro \
  -v /var/lib/lxcfs/proc/stat:/proc/stat:ro \
  -v /var/lib/lxcfs/proc/uptime:/proc/uptime:ro \
  -v /var/lib/lxcfs/proc/loadavg:/proc/loadavg:ro \
  -v /var/lib/lxcfs/proc/diskstats:/proc/diskstats:ro \
  -v /var/lib/lxcfs/proc/swaps:/proc/swaps:ro \
  -v /tmp/container-$i-cpu:/sys/devices/system/cpu:ro \
  ghcr.io/selkies-project/nvidia-egl-desktop:latest
```

> ⚠ **WARNING:** --network=host is POC only. In production with Cloudflare Tunnel, use --network=bridge and -p $NGINX_P:8080. Change CUDA_NVRTC_ARCH=89 to 100 on RTX 5090 production nodes.

11.3 — All 4 Containers (Loop)

```
TOTAL_VRAM_GB=24   # Change to 32 on RTX 5090

for i in 1 2 3 4; do
  NGINX_P=$((8080 + i))
  SELKIES_P=$((9080 + i))
  METRICS_P=$((19080 + i))
  DISPLAY_N=$((19 + i))
  CPUSET="$(( ($i-1)*4 ))-$(( ($i-1)*4+3 ))"
```

```
  VRAM_GB=$([ $i -le 2 ] && echo '4' || echo '8')
  SM_PCT=$(calc-sm-percent $VRAM_GB $TOTAL_VRAM_GB)
  MEM_BYTES=$((8*1024*1024*1024))
  SESSION_HOST="ws-$(openssl rand -hex 4)"
  TOKEN=$(openssl rand -hex 16)
  echo "Container $i → VRAM: ${VRAM_GB}GB | SM: ${SM_PCT}% | cpuset: $CPUSET | port:
$NGINX_P | token: $TOKEN"

  docker run -d \
    --name selkies-test-$i \
    --hostname $SESSION_HOST \
    --gpus all --cpus=4 --cpuset-cpus=$CPUSET --memory=8g \
    --ipc=host --network=host --tmpfs /dev/shm:rw \
    -e TZ=UTC -e DISPLAY=:$DISPLAY_N \
    -e DISPLAY_SIZEW=1920 -e DISPLAY_SIZEH=1080 -e DISPLAY_REFRESH=60 \
    -e SELKIES_ENCODER=nvh264enc \
    -e SELKIES_ENABLE_BASIC_AUTH=true \
    -e SELKIES_BASIC_AUTH_PASSWORD=$TOKEN \
    -e NGINX_PORT=$NGINX_P -e SELKIES_PORT=$SELKIES_P \
    -e SELKIES_METRICS_HTTP_PORT=$METRICS_P \
    -e PASSWD=mypasswd \
    -e CUDA_VISIBLE_DEVICES=0 -e CUDA_NVRTC_ARCH=89 \
    -e __NV_PRIME_RENDER_OFFLOAD=1 -e __GLX_VENDOR_LIBRARY_NAME=nvidia \
    -e CUDA_DEVICE_MEMORY_LIMIT_0=${VRAM_GB}192m \
    -e CUDA_DEVICE_SM_LIMIT=$SM_PCT \
    -e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT="0=${VRAM_GB}G" \
    -e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=$SM_PCT \
    -e CUDA_MPS_ENABLE_PER_CTX_DEVICE_MULTIPROCESSOR_PARTITIONING=1 \
    -e CONTAINER_MEMORY_LIMIT_BYTES=$MEM_BYTES \
    -e SELKIES_TURN_HOST=<TURN_IP> -e SELKIES_TURN_PORT=3478 \
    -e SELKIES_TURN_USERNAME=<TURN_USER> -e SELKIES_TURN_PASSWORD=<TURN_PASS> \
    -e SELKIES_TURN_PROTOCOL=tcp \
    -v /mnt/nfs/users/testuser$i:/home/ubuntu \
    -v /usr/lib/libvgpu.so:/usr/lib/libvgpu.so \
    -v /usr/lib/fake_sysconf.so:/usr/lib/fake_sysconf.so \
    -v /tmp/vgpulock-$i:/tmp/vgpulock \
    -v /usr/bin/nvidia-smi:/usr/bin/nvidia-smi.real \
    -v /tmp/nvidia-smi-wrapper:/usr/bin/nvidia-smi \
    -v /usr/bin/passwd:/usr/bin/passwd.real \
    -v /tmp/passwd-wrapper:/usr/bin/passwd \
    -v /tmp/supervisord-hami.conf:/etc/supervisord.conf \
    -v /tmp/bash.bashrc.orig:/etc/bash.bashrc \
    -v /var/lib/lxcfs/proc/cpuinfo:/proc/cpuinfo:ro \
    -v /var/lib/lxcfs/proc/meminfo:/proc/meminfo:ro \
    -v /var/lib/lxcfs/proc/stat:/proc/stat:ro \
    -v /var/lib/lxcfs/proc/uptime:/proc/uptime:ro \
    -v /var/lib/lxcfs/proc/loadavg:/proc/loadavg:ro \
    -v /var/lib/lxcfs/proc/diskstats:/proc/diskstats:ro \
    -v /var/lib/lxcfs/proc/swaps:/proc/swaps:ro \
    -v /tmp/container-$i-cpu:/sys/devices/system/cpu:ro \
    ghcr.io/selkies-project/nvidia-egl-desktop:latest
done
```

STEP 12 Validation Checklist

Run after every fresh setup. All items must pass before accepting user traffic.

12.1 — Host-Level Checks

```
# All containers running
docker ps --format 'table {{.Names}}\t{{.Status}}'

# Ports listening
ss -tlnp | grep nginx
# Expected: 0.0.0.0:8081, 8082, 8083, 8084

# MPS running
sudo systemctl status cuda-mps | grep Active

# lxcfs running
sudo systemctl status lxcfs | grep Active

# NFS mounted
mount | grep nfs
```

12.2 — Per-Container Automated Check

```
for i in 1 2 3 4; do
  echo "--- Container $i ---"
  docker exec selkies-test-$i bash -c '
    echo "hostname:  $(hostname)"
    echo "nproc:     $(nproc)"
    echo "ram:       $(free -h | awk "/^Mem/ {print \$2}")"
    echo "cpu_online: $(cat /sys/devices/system/cpu/online)"
    echo "ld_preload: $LD_PRELOAD"
  '
  VRAM_GB=$([ $i -le 2 ] && echo '4' || echo '8')
  docker exec selkies-test-$i python3 -c "
import struct
with open('/tmp/cudevshr.cache','rb') as f: d=f.read(4096)
limit=struct.unpack_from('<Q',d,1600)[0]
sm=struct.unpack_from('<Q',d,1728)[0]
print(f'  vram_limit: {limit/1024**3:.1f}GB  sm_limit: {sm}%')" 2>/dev/null || echo '
cache: not yet created (normal)'
done
```

12.3 — In-Container Validation (run in browser Konsole)

Command	Expected Result	What it validates
hostname	ubuntu@ws-xxxxxxxx	Random hostname, no host leakage
nproc	4	CPU cpuset enforced and visible
free -h	Mem: 8.0Gi	RAM limit enforced and visible
cat /sys/devices/system/cpu/online	0-3 (or 4-7, 8-11, 12-15)	CPU sys files correct
echo $LD_PRELOAD	/usr/lib/libvgpu.so /usr/lib/fake_sysconf.so	Both interceptors loaded

Command	Expected Result	What it validates
nvidia-smi (Memory-Usage column)	xxMiB / 4096MiB	VRAM limit displayed correctly
KDE → About this System → Memory	8.0 GiB	fake_sysconf.so working in KDE
KDE → About this System → Processor	Not shown / 4 cores	CPU sys files working in KDE
df -h /home/ubuntu	~15G (or 5G POC)	NFS home mounted correctly

12.4 — HAMi VRAM Enforcement Test (once torch is installed)

```
# Install torch (first time only — persists in NFS home)
pip install torch --index-url https://download.pytorch.org/whl/cu128 -q --break-system-
packages

python3 -c "
import torch
total = torch.cuda.get_device_properties(0).total_memory
print(f'Reported VRAM: {total/1024**3:.1f}GB')   # Must match tier limit
try:
    t = torch.zeros(int(total*1.5/4), dtype=torch.float32, device='cuda')
    print('FAIL: over-limit allocation succeeded')
except RuntimeError:
    print('PASS: over-limit allocation blocked')
try:
    t = torch.zeros(int(total*0.7/4), dtype=torch.float32, device='cuda')
    print('PASS: normal allocation succeeded')
except RuntimeError:
    print('FAIL: normal allocation blocked')
"
```

Appendix A — Tier Resource Configurations

The orchestrator maps subscription tier to resource parameters at container launch time.

Tier	CPUs	RAM	VRAM	SM% (RTX 4090)	SM% (RTX 5090)	Session Type
Starter	2	4GB	None	—	—	Stateful GUI
Standard	4	8GB	None	—	—	Stateful GUI
Pro	4	8GB	4GB	16%	12%	Stateful GUI
Power	6	16GB	8GB	33%	25%	Stateful GUI
Max	8	24GB	16GB	66%	50%	Stateful GUI
Full Node	16	64GB	24/32GB	100%	100%	Stateful GUI
Ephemeral CPU	2-4	4-8GB	None	—	—	CLI/Jupyter
Ephemeral GPU	4	8GB	4-8GB	16-33%	12-25%	CLI/Jupyter

Appendix B — Known Limitations

Limitation	Cause	Mitigation
nvidia-smi shows full GPU VRAM	NVML queries kernel driver — bypasses all user-space interception	nvidia-smi wrapper reads CUDA_DEVICE_MEMORY_LIMIT_0 env var and replaces total VRAM in output
Memory bandwidth shared	Consumer GPU (no MIG support on RTX)	SM proportional allocation minimises contention. Disclosed in ToS as shared-GPU limitation
--network=host (POC)	WebRTC requires direct port access over Tailscale	Production: --network=bridge + Cloudflare Tunnel
df shows overlay + NFS paths	Kernel-level mounts always visible	Use abstract NFS paths with no user-identifying info
lxcfs /sys filtering	lxcfs doesn't filter /sys by cpuset	Static per-container CPU files mounted at /sys/devices/system/cpu
Container startup ~15-25s	KDE Plasma initialisation time	Pre-warm pool for production UX

Appendix C — Troubleshooting

Symptom	Cause	Fix
selkies-gstreamer exit 139 in logs	Both libvgpu.so AND fake_sysconf.so in selkies-gstreamer LD_PRELOAD	Remove fake_sysconf.so from selkies-gstreamer — libvgpu.so only
LD_PRELOAD not set in Konsole	Injected in profile.d instead of bash.bashrc	Ensure /etc/bash.bashrc has the injection block
VRAM shows 23.5GB in ctypes test	ctypes.CDLL uses RTLD_LOCAL — bypasses LD_PRELOAD	Use torch for VRAM tests. ctypes always shows physical GPU.
nvidia-smi segfault	LD_PRELOAD + NVML conflict	Ensure nvidia-smi wrapper strips LD_PRELOAD before calling real binary
KDE RAM still shows host value	CONTAINER_MEMORY_LIMIT_BYTES env var missing	Verify -e CONTAINER_MEMORY_LIMIT_BYTES=$MEM_BYTES in docker run
nproc shows 32	--cpuset-cpus not in docker run	Add --cpuset-cpus=0-3 (or appropriate range) to docker run
KDE CPU shows host cores	/tmp/container-$i-cpu not mounted	Verify -v /tmp/container-$i-cpu:/sys/devices/system/cpu:ro in docker run
Container not accessible in browser	selkies-gstreamer in FATAL state	Check docker logs <name> --tail 30 — look for crash reason
HAMi not enforcing limits	libvgpu.so at wrong path	Must be /usr/lib/libvgpu.so — not /usr/local/lib/